

课程笔记

IAM for AI: 保护并面向未来设计智能体系统的 四步 IAM 成熟度模型

sss & Codex

2026 年 6 月 5 日



视频频道: IBM Technology

发布日期: 2026-04-12

视频时长: 00:14:38

视频链接: <https://www.youtube.com/watch?v=e8ela6puxig>

字幕来源: YouTube 英文自动字幕; 正文为中文教学化重写。

目录

1 为什么 Agentic IAM 需要成熟度模型	3
1.1 课程目标：用 IAM 保护 AI 代理与代理式系统	3
1.2 成熟度模型的直觉：从零散能力走向可运营控制	3
1.3 本章小结	4
2 四类核心风险：成熟度模型要解决什么	5
2.1 风险一：问责从项目团队转向代理本身	5
2.2 风险二：最小权限失效会制造过度授权代理	6
2.3 风险三：滥用既可能出于恶意，也可能来自开发失误	7
2.4 风险四：数据保护同时约束用户与代理	8
2.5 本章小结	8
3 Step 1 - Ad Hoc：从无治理状态看见真实风险	9
3.1 Ad Hoc 的含义：系统已经在跑，治理还没有跟上	9
3.2 它暴露出的盲点：团队不知道系统正在怎样行动	9
3.3 本章小结	10
4 Step 2 - Foundation：建立安全且可审计的代理访问	10
4.1 Foundation 的目标：最小可用控制层	10
4.2 控制一：给代理分配非人类身份	12
4.3 控制二：建立委托与 OBO 链路	12
4.4 控制三：SIEM 日志、审计与合规证据	13
4.5 本章小结	14
5 第三步：Enhanced，把代理纳入一等身份治理	14
5.1 从已有非人类身份走向一等身份主体	14
5.2 临时凭证：凭证只服务当前任务	16
5.3 细粒度上下文访问：把权限贴到任务路径上	16
5.4 实时检测：从事后审计走向运行时发现	18
5.5 本章小结	18
6 第四步：Adaptive 的持续认证、风险感知与实时撤销	19
6.1 为什么最高阶段必须是自适应	19
6.2 持续识别与持续认证：每一道敏感门都要验工牌	19

6.3	风险感知认证：敏感数据会抬高门槛	20
6.4	实时撤销：检测之后必须能阻断	21
6.5	本章小结	22
7	四步控制如何回到四类风险	22
7.1	成熟度是累积阶梯	22
7.2	问责：把行为贴回具体代理与凭证	23
7.3	最小权限：让代理只能做当前任务允许的事	23
7.4	防滥用：把观察、检测和撤销接成响应链	24
7.5	保护数据：把身份传播、委派与风险判断合在访问边界上	24
7.6	本章小结	24
8	总结与实施路线	24
8.1	从基础开始，沿成熟度模型提高	24
8.2	实践中的优先级与风险	25
8.3	本章小结	26

1 为什么 Agentic IAM 需要成熟度模型

1.1 课程目标：用 IAM 保护 AI 代理与代理式系统

这段 IBM Technology 的视频题为 *IAM for AI: 4 Steps to Secure and Futureproof Agentic Systems*。开场给出的任务很清楚：讨论面向 AI 代理和代理式系统的高级身份与访问管理策略，并用一个四步体系帮助团队提升环境成熟度、管理面向代理式系统的 IAM、让 AI 系统具备更长远的安全适应能力。这里的 IAM，即身份与访问管理，关心三个基础问题：某个行动者是谁，它能访问哪些资源，以及这些访问在什么条件下成立。

视频一开始就把对象限定在“AI agents”和“agentic systems”。AI 代理可以理解为一个能够调用工具、访问系统、执行步骤的软件行动者；代理式系统则是由一个或多个代理、工作流、工具调用和资源访问组成的动态执行环境。用生活中的比喻说，单个代理像一名拿着临时工牌去办事的工作人员，代理式系统像一条会根据任务变化自动调度人员、路线和工具的办事流程。安全问题随之变得更硬：系统需要知道每一步是谁做的、代表谁做的、凭什么权限做的、触达了哪些数据。

术语边界：面向代理式系统的 IAM

面向代理式系统的 IAM 指的是把传统 IAM 控制扩展到代理身份、委派链路、凭证生命周期、运行时上下文和数据访问决策。它关注一个可执行链条：代理以什么身份进入系统，拿到什么凭证，代表哪个用户或哪个实体行动，在什么上下文中访问哪类资源，最后留下什么审计证据。

这一讲的框架也提醒读者一个容易被忽略的事实：AI 安全讨论经常停留在模型输出、提示词和内容风险上，IAM 看到的是行动权限。只要代理可以调用 API、读取数据库、发起流程或连接业务系统，它就已经进入访问控制的范围。此时，提示词写得再克制，也无法替代身份、授权、审计和撤销这些工程控制。

1.2 成熟度模型的直觉：从零散能力走向可运营控制

讲者随后解释四步成熟度模型的来源。视频明确提到，成熟度模型的思路来自能力成熟度模型，时间线包括三个事实：1986 年，美国国防部赞助了相关工作；模型关注系统能力如何从几乎没有控制逐步提升到高级状态；1991 年，这套思路成形并逐渐成为成熟度模型的常用参照。

成熟度模型的价值在于排序。很多团队面对代理式 AI 时会列出一长串愿望：要有身份、要有日志、要有限权、要能发现异常、要能撤销凭证。愿望清单很快会变成噪声，因为每一项都重要，工程团队却必须决定先做什么、后做什么、哪些控制依赖前置能力。成熟度模型把这种噪声压缩成阶梯：每一级引入一组新的运营控制，让系统从低治理状态走向可审计、可约束、可动态响应的状态。

可以用一个简单公式表达这种递进关系：

$$C_{t+1} = C_t + \Delta C$$

式中, C_t 表示当前成熟度阶段已经具备的可运营能力; C_{t+1} 表示进入下一阶段后的能力; ΔC 表示新增的一组控制, 例如代理身份、委派链路、短期凭证、上下文访问判断或实时撤销。这个公式的含义很朴素: 成熟度提升依靠一组能落地、能运行、能被审计的控制增量完成。

为什么要先讲成熟度

代理式系统的路径会变化, 执行步骤也可能每次不同。团队如果直接追逐最高级控制, 很容易在没有身份锚点、没有委派证据、没有基础日志的情况下谈实时风控。成熟度模型先建立阶梯, 再把每一阶和具体风险对应起来, 这样安全建设才有依赖顺序。

因此, 视频在进入四个成熟度步骤之前, 先要求读者理解风险。这个安排很关键: 成熟度步骤承担的是解法角色, 风险承担的是问题角色。后续的 Ad Hoc、Foundation、Enhanced、Adaptive 四级成熟度, 都应回到同一个问题上衡量: 它们分别减少了哪些身份风险、权限风险、滥用风险和数据风险。

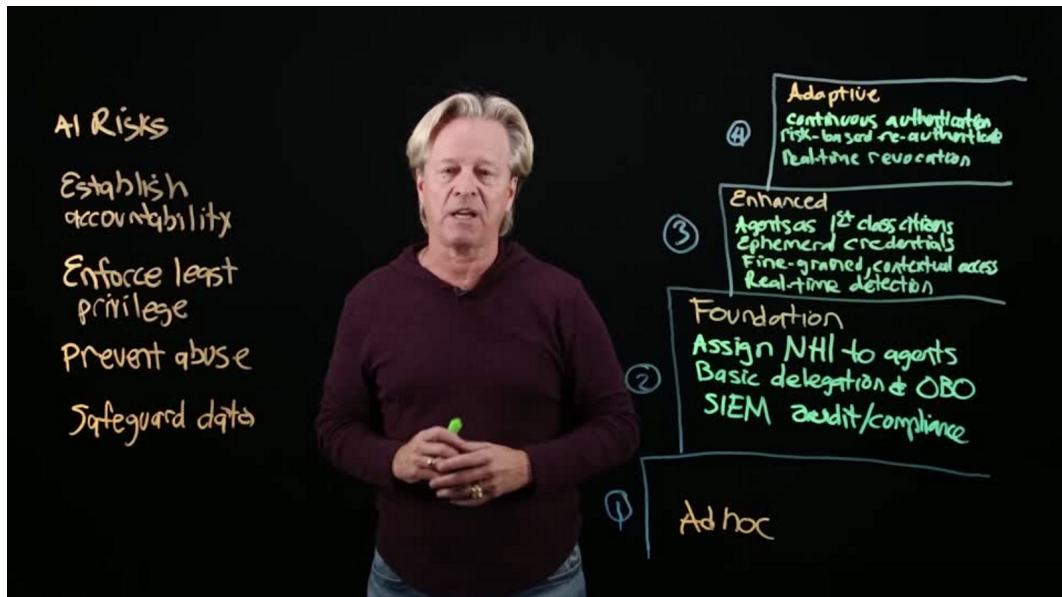


图 1: 完整成熟度模型: 左侧列出四类 AI 风险, 右侧用四级台阶串起 Ad Hoc、Foundation、Enhanced 和 Adaptive。¹

1.3 本章小结

本章建立了这份讲义的起点: 源视频讨论的是用 IAM 保护 AI 代理和代理式系统, 并通过四步成熟度体系让环境逐级成熟。能力成熟度模型的历史线索提供了一个工程化直觉: 把安全建设拆成可累计的控制增量。接下来要先看四类核心风险, 因为没有清晰的风险轴, 成熟度模型就会退化成一堆互相抢优先级的安全功能。

¹视频画面时间区间: 00:14:05–00:14:15。

2 四类核心风险：成熟度模型要解决什么

2.1 风险一：问责从项目团队转向代理本身

讲者列出的第一类风险是 accountability，即问责或可追责性。传统应用常见的做法，是由一个项目团队构建应用，再用 API key、功能账号或系统账号把应用连接到其他系统、流程和数据。这个模式在传统系统里经常勉强够用，因为执行路径较稳定，项目边界较明确，问题发生后往往能追到某个团队或某个应用。

代理式系统改变了问责的重心。代理可能根据任务动态选择工具、组合步骤、访问多个系统；非确定性意味着同一个高层目标在不同运行中可能产生不同路径。此时，只把责任挂在项目团队名下会留下关键空白：哪一个代理执行了动作，动作发生在什么上下文中，它代表哪个用户或实体，使用了哪类凭证，连接了哪个资源。

问责空白的真实后果

如果审计日志只能显示“某项目的系统账号读取了客户数据”，调查人员仍然缺少关键证据：具体代理、触发输入、代理目标、委派关系和凭证来源。代理越能自主规划，问责越需要贴近代理和 AI 行动者本身。

可以把传统系统想象成一辆固定线路公交车，司机、路线和站点基本稳定；代理式系统更像临时调度车辆，会根据目标、交通和任务临时改路线。固定线路时代，用车队编号追责可能还能定位问题；动态调度时代，需要知道哪辆车、哪名司机、哪张派车单、哪段路线出现了异常。IAM 在这里提供的就是这套身份与证据链。

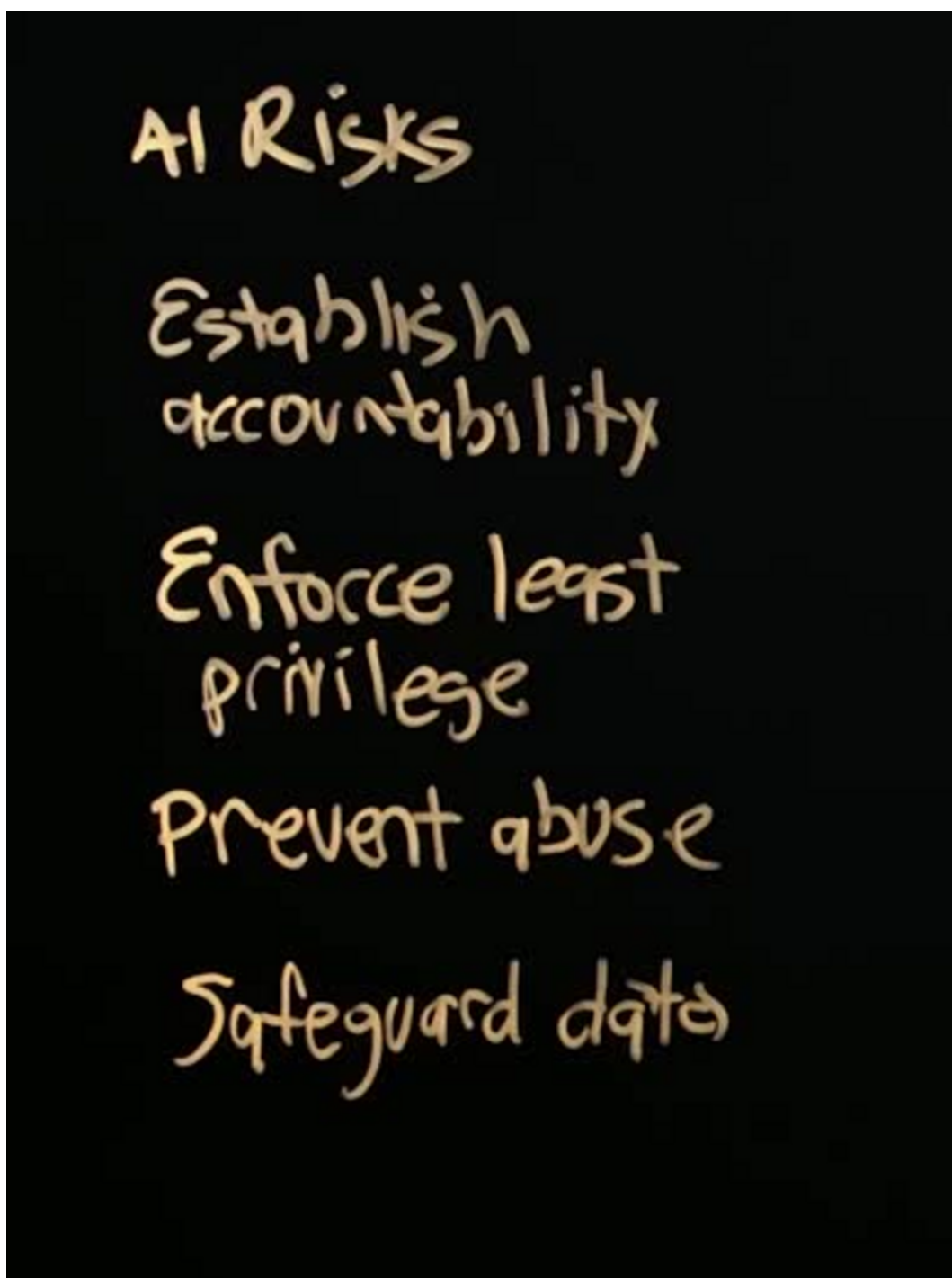


图 2: 讲者在白板上列出的四类 AI 风险: 建立可追责性、执行最小权限、防止滥用、保护数据。²

2.2 风险二: 最小权限失效会制造过度授权代理

第二类风险是 least privilege, 即最小权限。讲者强调, 团队应该避免 “super agents” 或 “super agency”: 代理获得过宽权限, 可以做很多与当前任务无关的事, 进而访问过多系统。这里可以把 “超级代理” 理解为过度授权代理: 它看起来很方便, 因为一个代理几乎什么都能连、什么都能做; 它也很危险, 因为攻击者或错误流程只要接管这一个代理, 就可能横向触达大量资源。

²视频画面时间区间: 00:03:55–00:04:05。

最小权限可以用一个简洁不等式表示：

$$P_{\text{granted}} \leq P_{\text{required}}$$

式中， P_{granted} 表示系统实际授予代理的权限集合； P_{required} 表示代理完成当前任务所需的最小权限集合；符号 $\leq$ 表示授予权限不应超过任务所需权限。这里的“大小”可以理解为权限集合的覆盖范围，例如可访问资源数量、可执行操作类型、可持续时间和可使用上下文。

风险出现在权限差额变大时：

$$\Delta P = P_{\text{granted}} - P_{\text{required}}$$

式中， ΔP 表示超出任务需要的权限余量； P_{granted} 仍表示已授予权限； P_{required} 仍表示完成任务所需权限。 ΔP 越大，代理被误用、被诱导或被攻击后造成的影响面越大。若出现 $P_{\text{granted}} \gg P_{\text{required}}$ ，其中 $\gg$ 表示“远大于”，系统就接近讲者所警告的超级代理状态。

为什么代理让最小权限更难

传统应用的访问路径通常比较稳定，权限可以按应用长期配置。代理式系统的任务路径每次可能变化，代理也可能临时选择工具和资源。最小权限因此需要贴近任务、时间、上下文和目标动作，不能只靠长期有效的宽泛 API key。

讲者给出的目标非常具体：代理应聚焦单一任务，只拿完成这个任务所需的最少访问权和最少特权。这个原则看似朴素，却是后面短期凭证、细粒度上下文访问和实时撤销的根。没有最小权限，后续控制会变成补救；有了最小权限，后续控制才是在缩小权限窗口和缩短风险时间。

2.3 风险三：滥用既可能出于恶意，也可能来自开发失误

第三类风险是 prevent abuse，即防止滥用。讲者特别把滥用分成两种来源。第一种是恶意来源：有人有意构建一个代理，让它去执行组织不希望发生的动作。第二种是无意来源：开发者认为自己在实现一个很好的 AI 用例，却把代理连接到了某个系统，结果引入了数据泄露、越权访问或不该发生的业务操作。

这一区分很重要，因为安全控制如果只围绕恶意攻击设计，就会漏掉大量真实事故。代理式系统里的无意滥用并不少见：一个内部助手本来只想查询工单，却被接入了客户数据库；一个自动化流程本来只想总结邮件，却被授予了发送、转发和下载附件的权限；一个数据分析代理本来只想读取聚合指标，却在运行中触达了明细级敏感数据。开发者的意图可能是提高效率，系统结果却可能是越权执行。

防滥用的工程含义

防滥用需要同时覆盖意图和错误。意图问题需要身份、授权、检测和撤销；错误问题需要最小权限、上下文约束、日志和异常发现。代理式系统的风险来源包括坏人，也包括好人把代理连到了错误的资源、授予了过大的权限、缺少了运行时边界。

因此，滥用风险与前两个风险相连。问责回答“谁做的”，最小权限限制“最多能做什么”，防滥用继续追问“系统能否发现正在做错事，并及时阻断”。这就是成熟度模型后面要引入 SIEM、实时检测和实时撤销的原因。

2.4 风险四：数据保护同时约束用户与代理

第四类风险是 safeguard data，即保护数据。讲者把场景说得很具体：代理可能查询数据、取回信息，也可能发起某个业务流程。只要代理代表某个用户行动，系统就要同时检查两件事：这个用户是否有权访问该数据，以及这个代理是否被允许访问该数据。

这可以理解为双钥匙开门。第一把钥匙属于用户，证明用户本人对数据有访问资格；第二把钥匙属于代理，证明这个代理在当前任务、当前上下文和当前授权链路中可以触达数据。只有用户有权限还不够，因为代理可能超出任务边界；只有代理有权限也不够，因为代理不能借自己的系统权限替用户绕过数据边界。

可以把数据访问决策写成一个直观的合取关系：

$$A_{\text{data}} = A_{\text{user}} \wedge A_{\text{agent}}$$

式中， A_{data} 表示最终是否允许访问数据； A_{user} 表示用户侧授权判断是否通过； A_{agent} 表示代理侧授权判断是否通过； $\wedge$ 表示逻辑“且”，也就是两个条件都成立时才允许访问。这个公式刻意简化了真实 IAM 系统中的上下文、风险和时间因素，但它准确保留了讲者强调的双重授权要求。

数据保护的未知盲点

很多团队会默认“代理代表用户，所以沿用用户权限即可”。这个默认值很危险。代理代表用户行动时，还需要检查代理自身是否被允许执行该类数据操作、是否在授权任务内、是否处在可信上下文中。缺少代理侧授权，系统就可能把用户权限放大成代理的通用通行证。

数据保护最终需要护栏来防止未授权访问。这里的护栏是一组可执行的访问判断：身份能传播，委派链能保留，代理身份能被识别，数据敏感度能参与决策，异常行为能被发现并阻断。后续成熟度步骤会逐步把这些护栏落到身份、凭证、日志、上下文访问和撤销机制上。

2.5 本章小结

本章把成熟度模型要解决的问题拆成四条风险轴：问责、最小权限、防止滥用和保护数据。问责要求系统能追到具体代理和 AI 行动者；最小权限要求代理只获得当前任务所需权限；防止滥用要求控制既覆盖恶意行为，也覆盖开发者无意引入的越权和泄露；保护数据要求用户授权与代理授权同时成立。后续四级成熟度模型应按这四条轴来评估：每升一级，系统都要更清楚谁在行动、更精确地限制权限、更早发现错误、更可靠地守住数据边界。

3 Step 1 - Ad Hoc：从无治理状态看见真实风险

3.1 Ad Hoc 的含义：系统已经在跑，治理还没有跟上

讲者把成熟度模型的最低台阶称为 Ad Hoc，也就是临时拼接、局部决策、缺少一致治理的状态。在这个阶段，团队通常已经开始构建 AI 代理和代理式系统：有人在做内部助手，有人在把模型接到工单、客户数据、代码仓库或业务流程，也有人为了展示 AI 能力而快速上线实验性应用。问题在于，交付压力先到，身份治理、权限边界、运行时可见性和审计证据经常滞后。

Ad Hoc 描述的是已经离开空白实验室的运行状态。更麻烦的地方在于，系统已经拥有行动能力，却还没有被当成需要治理的行动者来管理。一个代理可能调用 API、查询数据库、触发流程、代表用户提交请求，团队却没有认真回答几个基础问题：这个代理以什么身份运行，它连到了哪些系统，它能访问哪些资源，它的任务边界在哪里，失败或越权时由什么证据还原事实。讲者在 00:04:30-00:05:12 的核心提醒很直接：很多团队正在 AI 压力下搭建系统，却没有思考风险、系统运行方式以及它到底运行在什么资源之上。

最低台阶的真正危险

Ad Hoc 的风险远超过缺少高级安全功能。更关键的风险是系统已经能行动，组织却低估了这种行动能力需要身份、权限和证据来约束。一个没有审计链路的代理，像一名没有工牌、没有任务单、没有出入记录的外包人员进入办公楼；短期内它也许能把事情办完，一旦出现数据访问、流程误触发或横向移动，调查人员会发现关键事实缺失。

这里需要把 Ad Hoc 看成诊断基线，避免把它当作可以长期接受的成熟状态。它回答的是“团队现在离可控运行还有多远”。如果一个系统只能说明“有一个 AI 功能调用了某个接口”，却无法说明“哪个代理、在什么任务下、代表谁、凭什么权限、访问了什么资源、留下了什么日志”，那它就在问责、最小权限、防滥用和数据保护四条风险轴上同时暴露缺口。

用一个工程类比来讲，Ad Hoc 像临时把多台机器接进生产网络：每台机器都能发包、读写服务、触发作业，运维团队却还没有资产登记、访问策略、日志采集和告警规则。网络本身会运行，业务也可能短期可用，但系统越能自动化，未知盲点越多。代理式系统也一样，低运行理解度会直接扩大风险面，因为代理的路径可能随任务变化，风险也会随运行时上下文变化。

Ad Hoc 的核心定义

Ad Hoc 是代理式 IAM 成熟度模型的第一步，也是最低成熟度台阶。它描述的是这样一种状态：团队在 AI 交付压力下构建代理和系统，已有基本运行能力，却缺少有意识的身份治理、权限设计、运行时理解、日志证据和审计机制。

3.2 它暴露出的盲点：团队不知道系统正在怎样行动

Ad Hoc 最值得警惕的地方，是团队经常把“能跑起来”误当成“能安全运行”。讲者提到，团队可能只是想以非常基础的方式运行系统，尚未思考风险。这里的风险来自三个具体盲点的叠加，绝非抽象恐惧。

第一，缺少身份锚点。代理如果只共享一个项目级 API key、功能账号或系统账号，日志里

看到的是账号活动，难以看到代理活动。问责链会停在“某个系统账号做了事”，无法继续定位到“哪个代理为哪个任务做了事”。第二，缺少运行时理解。代理式系统可能根据提示、工具返回、模型判断和上下文变化选择不同步骤，团队如果没有观测和日志，很难知道一次运行实际经过了哪些资源。第三，缺少治理边界。没有明确的委托、权限范围和证据链，代理就可能在用户、系统、资源之间形成模糊的行动通道。

这也是本节的“未知盲点”：Ad Hoc 阶段最缺的东西经常不会在上线当天暴露。系统第一次演示时可能表现正常，第一次内部试点也可能看起来有效。真正的问题会在代理接入更多资源、处理更敏感数据、执行更长链路任务时出现。到那时，组织才发现自己缺少三个基础事实：谁在行动，权限从哪里来，事后如何还原。

为什么 AI 压力会制造 Ad Hoc

AI 项目容易出现“先接上工具再补治理”的顺序，因为业务方希望尽快看到自动化效果，开发方希望快速验证用例，安全团队往往在系统连到真实资源之后才进入。对传统软件来说，这种顺序已经有风险；对代理式系统来说，风险会更大，因为代理既能根据上下文改变路径，又能代表用户或系统身份访问资源。

因此，Ad Hoc 阶段的正确作用是让团队感到不舒服。它把成熟度模型的起点压得很低：系统可能已经运行，却还没有可靠问责；代理可能已经访问资源，却还没有明确身份；任务可能已经触发动作，却还没有可审计证据。只有把这种不适感说清楚，下一步 Foundation 才有工程意义。

3.3 本章小结

本节覆盖了成熟度模型的第一步：Ad Hoc。它是最低台阶，描述团队在 AI 压力下快速构建代理，却缺少风险思考、运行时理解和治理机制的状态。Ad Hoc 的危险在于系统已经有行动能力，组织却无法稳定回答“谁行动、代表谁、凭什么权限、访问什么资源、留下什么证据”。它属于诊断基线，不具备成熟能力；它提醒团队，若没有身份、委托、日志和审计，代理式系统很可能已经在没有问责能力的情况下运行。

4 Step 2 - Foundation：建立安全且可审计的代理访问

4.1 Foundation 的目标：最小可用控制层

进入第二步后，讲者把下一个成熟度层级称为 Foundation。Foundation 的目标很朴素：放入最低限度的控制，让系统至少按照组织期望的方式运行。这里的“按照期望运行”包括两层含义。第一，代理访问应当是安全的，系统知道代理是谁、权限从哪里来、能触达什么资源。第二，代理访问应当是可审计的，系统能在事后提供日志、事件、委托链和合规证据。

字幕中把这一段识别成“secure and audible agent access”，结合上下文与画面中的 Foundation 条目，应理解为 secure and auditable agent access，即“安全且可审计的代理访问”。同一画面还显示了“SIEM audit/compliance”，这支持把字幕里的“SIM”修正为 SIEM。SIEM 是 Security Information and Event Management，中文可译为安全信息与事件管理，用于日志采集、事件汇总、检测、审计和合规支持。

Foundation 解决的三个基础问题

Foundation 仍处在基础控制层，距离最高级运行时防线还有明显差距。它先回答三个基础问题：谁是代理，权限如何沿流程委托，行动之后有什么证据可查。换句话说，它把代理从一个模糊的软件动作，变成一个有身份、有授权关系、有日志痕迹的受治理行动者。

Foundation 仍然很基础。它提供的是身份锚点和可追踪证据，强运行时执行、实时异常阻断、持续认证和实时撤销会在后续 Enhanced 与 Adaptive 阶段加强。这个边界必须说清楚，否则团队容易把“有日志”误判为“已经能实时控制风险”。日志能帮助发现和还原，执行层控制才能在运行中收紧、拒绝或撤销访问。

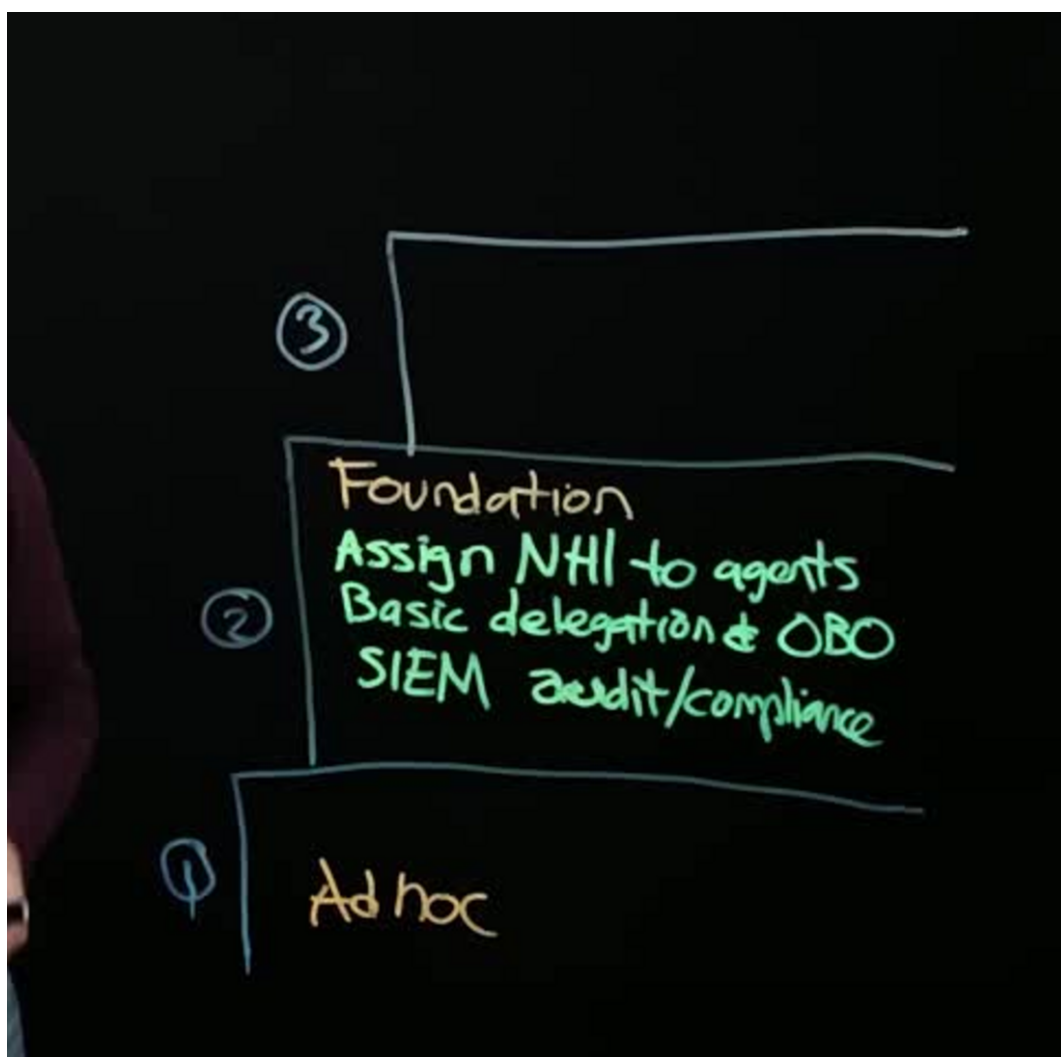


图 3: Foundation 层的完整可见状态：为代理分配非人类身份，建立基础委派与 OBO，并通过 SIEM 形成审计与合规证据。³

³视频画面时间区间：00:07:25–00:07:35。

4.2 控制一：给代理分配非人类身份

Foundation 的第一项控制，是给代理分配非人类身份。非人类身份指分配给软件行动者的身份，例如服务、工作负载、自动化脚本、机器人和 AI 代理。对代理式系统来说，这个身份至少要代表三件事：代理本身、代理执行的动作、代理承担的任务。

这一步是问责的第一个锚点。没有代理身份，后续日志只能记录某个系统账号或项目凭证的活动；有了代理身份，审计人员才有机会追踪到具体代理。例如，一个“合同摘要代理”和一个“客户退款代理”即使运行在同一平台上，也应该拥有可区分的身份。前者可能只需要读取合同文档并生成摘要，后者可能需要读取订单、调用退款流程、写入工单备注。把它们都藏在同一个共享账号后面，等于把不同风险等级的行动者揉成一个影子。

非人类身份与传统服务账号的关系

传统服务账号通常服务于应用、后台进程或系统集成。代理身份可以借鉴这种做法，但需要更细的语义：它不仅说明“哪个程序在跑”，还要说明“哪个代理为了哪个任务在行动”。对代理式系统来说，身份越贴近任务和行动，问责链越清楚，最小权限也更容易落地。

可以用一个简单关系式表达 Foundation 对身份的最低要求：

$$I_{\text{agent}} = (Ag, T, Act)$$

式中， I_{agent} 表示代理身份记录； Ag 表示具体代理； T 表示该代理当前承担的任务； Act 表示代理执行的动作集合。这个式子属于教学抽象，不能直接当作产品协议：一个可治理的代理身份不应只保存名字，还要能关联任务和动作，否则审计只能看到“某代理出现过”，无法理解“它为什么行动、行动范围是什么”。

4.3 控制二：建立委托与 OBO 链路

有了代理身份，Foundation 的第二项控制是建立基础委托与 on-behalf-of 链路。Delegation，即委托，表示一个实体把受控权限传递给另一个实体，让后者在限定条件下执行任务。OBO，即 on behalf of，表示代理代表某个发起用户行动，同时保留这个用户的身份上下文。

在用户提示触发的系统里，链路通常从用户开始。用户输入目标，代理式系统接收目标，再把任务分配给具体代理，代理访问资源并留下安全事件。可以把这条链写成：

$$U \rightarrow S \rightarrow Ag \rightarrow Res \rightarrow E_{\text{SIEM}}$$

式中， U 表示发起请求的用户； S 表示代理式系统； Ag 表示执行具体任务的代理； Res 表示被访问的资源； E_{SIEM} 表示写入 SIEM 的安全事件。箭头 $\rightarrow$ 表示身份上下文、授权关系和行动证据沿流程传递。这个链条的意义在于，资源访问记录会从“某个系统调用了资源”的粗粒度事件，升级为“某个用户通过某个代理式系统委托某个代理访问某个资源，并产生可审计事件”的细粒度证据。

这条链也适用于自主系统，只是起点可能不再是正在交互的人类用户。自主代理可能由计划任务、业务规则、传感器事件或另一个系统触发。即便没有活跃的人类用户，仍然存在实体到实

体的授权传递：某个自动化触发器把受限任务交给代理，代理凭自己的非人类身份访问资源，系统仍要记录权限来源、任务目标和资源动作。讲者强调，即使在代理自行行动的系统里，也需要维护实体之间的权利委托。

OBO 容易被误解成“拿用户权限直接跑”

OBO 的重点是保留用户上下文并受控委托，严禁把用户的全部权限复制给代理。正确的链路应当同时检查用户是否能访问目标资源、代理是否被允许执行当前任务、委托是否仍在有效范围内。缺少这些检查，OBO 会退化成权限放大通道。

一个直观例子是差旅代理。员工要求代理预订机票，员工本人也许有权访问自己的差旅预算；代理则只应获得查询航班、创建审批单、读取该员工预算状态的任务权限。它不能因为“代表员工”就读取所有员工的预算，也不能因为“是差旅代理”就绕过审批流程。OBO 链路要把用户、代理、任务、资源和证据绑定在一起。

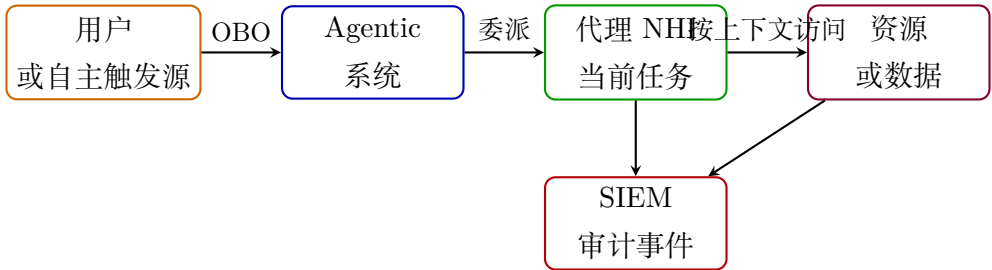


图 4: Foundation 层的委派链：发起身份、代理身份、资源访问和审计事件需要被连成一条可追踪链。

4.4 控制三：SIEM 日志、审计与合规证据

Foundation 的第三项控制，是把代理活动送入 SIEM，形成日志、审计和合规证据。SIEM 的全称是 Security Information and Event Management，中文是安全信息与事件管理。它通常负责汇聚系统日志、身份事件、访问事件、告警信号和合规记录，让安全团队能够检索、关联、检测和报告。

在 Foundation 阶段，SIEM 最直接的价值是证据。代理访问资源时，系统应记录代理身份、委托来源、用户上下文、目标资源、动作类型、时间戳和结果。这样的日志未必能在第一时间阻断风险，但它能让组织在事后回答关键问题：哪一个代理访问了资源，它代表谁，权限从哪里委托，访问是否符合策略，是否触发了异常模式，是否满足合规审计要求。

Foundation 的证据链

Foundation 的证据链可以概括为：代理身份说明“谁在行动”，委托和 OBO 说明“代表谁、权限从哪里来”，SIEM 事件说明“做了什么、何时做、结果如何”。这三类证据放在一起，才构成安全且可审计的代理访问。

这里要补上一个容易被忽略的未知盲点：Foundation 主要提供可见性、证据和追踪能力，尚未形成完整的运行时强制执行层。日志记录了事件，审计还原了事件，合规报告证明了流程存在；

但是，如果代理在运行中开始偏离任务，Foundation 本身未必能立即收紧权限或撤销凭证。后续 Enhanced 会把代理当作一等身份主体，引入更细的凭证和上下文访问；Adaptive 会进一步加入持续认证、风险感知和实时撤销。Foundation 是这些能力的地基，因为没有身份和日志，后面的实时控制也缺少判断依据。

可以把 Foundation 的输出看成一个最小可审计事件：

$$E_{\text{SIEM}} = (I_{\text{agent}}, D, U, Res, Act, t, result)$$

式中， E_{SIEM} 表示写入 SIEM 的安全事件； I_{agent} 表示代理身份； D 表示委托关系； U 表示用户或触发实体； Res 表示资源； Act 表示动作； t 表示事件发生时间； $result$ 表示动作结果，例如成功、拒绝、失败或触发告警。这个事件结构的价值在于，它把“代理做了某件事”变成可查询、可关联、可审计的记录。

4.5 本章小结

本节覆盖了成熟度模型的第二步：Foundation。它是从 Ad Hoc 走向可治理代理访问的最小控制层，目标是让系统具备安全且可审计的基本行为。Foundation 包含三项核心控制：给代理分配非人类身份，让身份代表代理、动作和任务；建立基础委托与 OBO 链路，让用户提示型系统和自主系统都保留权限来源；把代理活动写入 SIEM，形成日志、审计和合规证据。它回答“谁是代理、权限如何委托、行动后有什么证据”这三个问题。更强的运行时收紧、实时检测、持续认证和实时撤销会在后续成熟度层级出现；Foundation 的作用是先让系统有身份锚点和可追踪证据。

5 第三步：Enhanced，把代理纳入一等身份治理

5.1 从已有非人类身份走向一等身份主体

Foundation 阶段已经让代理拥有某种可追踪的非人类身份，并通过委派、on-behalf-of 关系和 SIEM 日志留下审计证据。Enhanced 阶段继续往前推进：IAM 系统开始把代理当作独立、可治理、可撤销的身份主体处理。讲者用“first-class citizens”描述这个变化，意思很具体：代理在身份治理系统里拥有自己的生命周期、策略、审计记录、治理责任、所有权归属和撤销路径，摆脱长期躲在项目 API key、功能账号或系统账号背后的粗粒度状态。

一等身份主体可以用员工身份作类比。一个员工入职时会被创建身份，换岗时会调整权限，离职时会撤销访问，日常行为会进入审计系统，并且有人对这个身份的合理使用负责。代理也需要类似的治理面：创建时要说明用途、所有者、可代表的用户或业务任务；运行时要绑定策略、资源范围和委派关系；审计时要能追到具体代理、具体凭证、具体任务和具体动作；退役或异常时要能停用身份并收回相关凭证。

一等身份主体的工程定义

在 Agentic IAM 中，代理成为一等身份主体，意味着 IAM 系统直接管理它的身份生命周期、访问策略、审计证据、治理责任、所有权和撤销机制。判断标准很朴素：安全团队能否回答“这个代理是谁、由谁负责、为哪个任务行动、拿了哪张凭证、访问了什么资源、异常时如何停用”。若这些问题只能追到项目账号或应用账号，治理仍停留在粗粒度阶段。

这个转折点的价值在于把“知道有代理在行动”升级为“能管理每一个代理怎样行动”。传统非人类身份常常服务于应用或工作负载，边界相对固定；代理式系统里的行动者会根据目标临时选择工具、资源和步骤。若 IAM 只给它一个泛化系统账号，后续策略很难分辨当前动作究竟属于合理任务、越权尝试，还是模型路径漂移带来的错误行为。

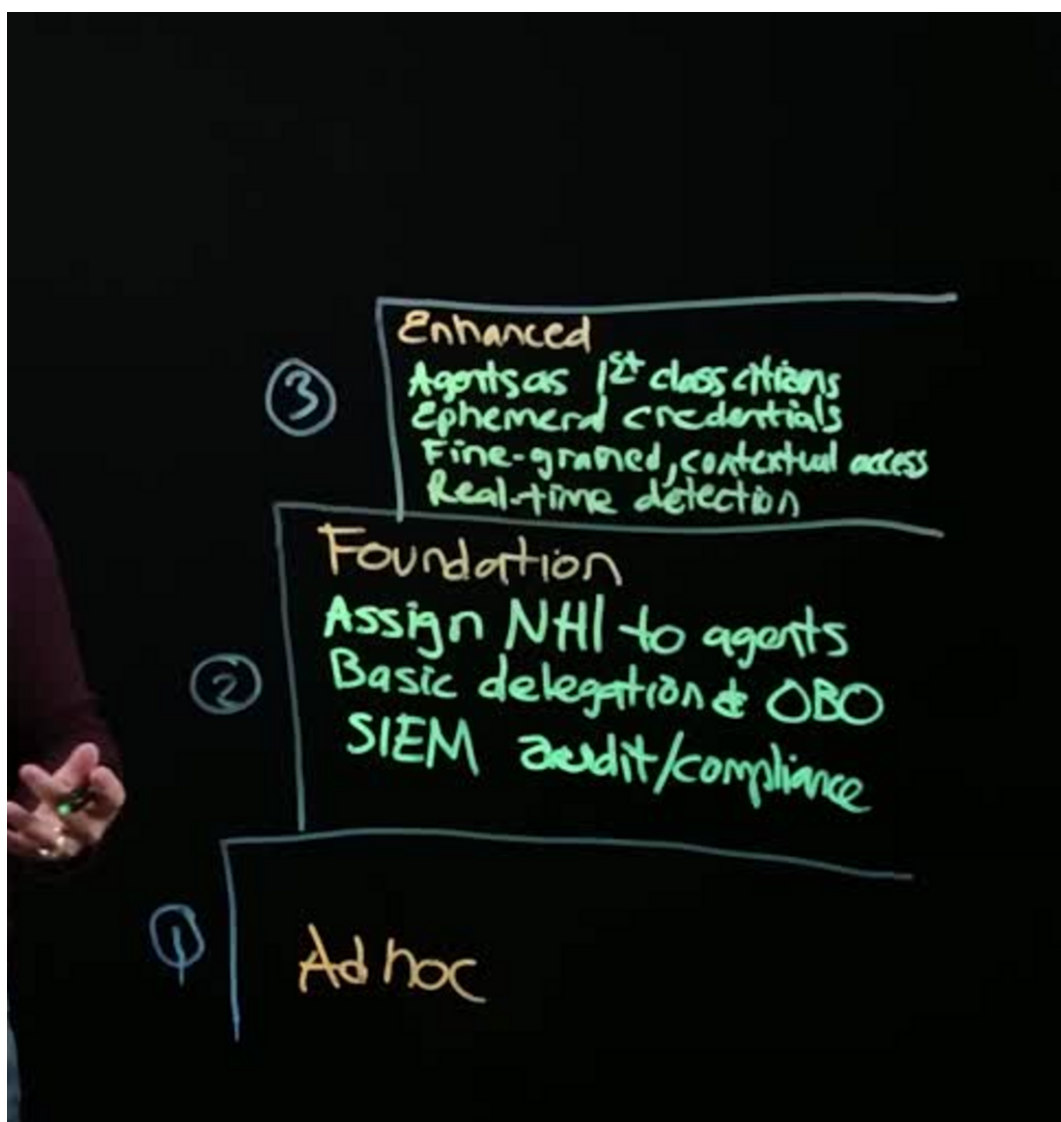


图 5: Enhanced 层的完整可见状态：代理成为一等身份对象，凭证变为任务级临时凭证，访问决策纳入细粒度上下文，并加入实时检测。⁴

⁴视频画面时间区间：00:10:25–00:10:35。

“有账号”距离“可治理身份”仍有距离

给代理分配一个系统账号只能提供最低限度的可见性。Enhanced 阶段要求更高：身份要能被生命周期管理、策略约束、审计解释、责任归属和异常撤销串起来。只登记账号名，缺少任务、所有者、凭证边界和撤销机制，会让一等身份沦为标签。

5.2 临时凭证：凭证只服务当前任务

代理成为一等身份主体之后，下一步是控制它拿到的凭证。讲者强调，人的身份和凭证通常更持久：员工账号、登录会话、企业设备证书都可能跨越多天、多周甚至更长周期。代理凭证应当更短，只围绕当前任务签发。代理要查一张工单，就给它查这张工单所需的凭证；代理要完成一次客户资料摘要，就让凭证覆盖这次摘要所需的资源和时间窗口。任务结束后，凭证应随之失效。

这背后的直觉像临时施工证。大楼不会因为某个外包工程师今天要修一台设备，就给他一张全年通行全楼层的门禁卡；更合理的做法是发一张当天有效、只允许进入指定楼层和设备间的施工证。代理凭证也应贴近任务，避免把一次性行动变成长期通行能力。

临时凭证的时间边界可以写成：

$$T_{\text{credential}} \leq T_{\text{task}} + \epsilon$$

其中， $T_{\text{credential}}$ 表示代理凭证的有效时长； T_{task} 表示当前任务从开始到结束的执行时长； ϵ 表示很小的运维宽限时间，例如日志落盘、结果提交或事务收尾所需的短暂缓冲；符号 $\leq$ 表示凭证有效时长应小于或等于任务时长加上这段宽限时间。这个公式的含义是：凭证不能明显长于任务。若任务已经结束，凭证仍能继续访问资源，风险窗口就被人为拉长。

短生命周期凭证压缩的是风险时间窗

最小权限通常关注“能访问什么”，临时凭证进一步关注“能访问多久”。同样的资源权限，持续十分钟和持续十个月的风险完全不同。凭证越短，攻击者复用泄露凭证的机会越少，错误任务继续扩散的时间也越短。

这里还需要区分身份和凭证。身份回答“这个代理是谁”，凭证回答“这个代理此刻凭什么访问”。代理身份可以长期存在，便于治理、归属和审计；代理凭证应按任务短期签发，便于限制权限窗口。把两者混在一起会制造治理盲点：若身份短到无法审计，问责会丢失；若凭证长到跨越多个任务，最小权限会失效。

5.3 细粒度上下文访问：把权限贴到任务路径上

Enhanced 阶段的第三个能力是细粒度、上下文相关的访问控制。传统应用经常使用稳定 API key 或长期应用访问权：应用拿到一组权限，经过周期性轮换后继续调用固定资源。这个模式在稳定系统里容易理解，因为应用路径相对可预测，权限集合也相对固定。代理式系统的难点在于非确定性：同一个目标可能在每次运行中选择不同工具、访问不同资源、走不同步骤，任务路径会随上下文变化。

比较维度	传统 API key / 稳定应用访问	代理式临时凭证 / 动态任务访问
有效期	通常较长，依赖轮换周期降低风险	贴近单次任务，任务结束后失效
权限边界	围绕应用或服务预先配置	围绕代理、任务、资源、上下文和目标动作实时收窄
路径假设	调用链相对稳定，资源范围可提前枚举	路径可能每次变化，需要按运行时变量判断
审计粒度	常追到应用、项目或系统账号	追到代理身份、任务、凭证、动作和资源

因此，访问决策要把更多变量放进策略面。一个足够直观的表达式是：

$$Decision = f(identity, task, resource, context, risk, time)$$

其中，*Decision* 表示访问决策，结果可以是允许、拒绝、要求更强认证或降级访问；*f* 表示策略决策函数，也就是组织把多个输入组合成访问判断的方法；*identity* 表示身份信息，包括代理身份、用户身份、委派关系和所有者；*task* 表示当前任务目标与任务边界；*resource* 表示目标资源，并且在具体策略里要连同目标动作一起理解，例如读取、写入、导出、删除、转发或触发业务流程；*context* 表示运行时上下文，例如来源系统、网络位置、工具链、会话状态和业务场景；*risk* 表示当前风险等级或异常信号；*time* 表示时间因素，包括任务时段、凭证剩余有效期和访问发生的具体时刻。

这个公式的重点在于“访问”从静态清单变成运行时判断。代理身份说明行动者是谁，任务说明为什么行动，资源说明触达对象，目标动作说明要做什么，上下文说明行动发生在哪里和怎样发生，风险说明当前是否偏离预期，时间说明凭证和任务是否仍处在允许窗口。少看任何一个变量，都会把动态任务路径压扁成粗糙的应用权限。

上下文访问就是运行时最小权限

细粒度上下文访问把最小权限落实到“这个代理、这个任务、这个资源、这个动作、这个上下文、这个风险水平、这个时间点”。它把关注点从长期应用授权收紧到当前任务现场，核心目标是让代理在当前任务里只拿到当前动作所需的访问能力。

一个具体例子能说明差异。若客服代理正在总结某个客户的最近工单，它可能需要读取该客户的工单标题、状态和处理记录。它此刻不需要导出全量客户表，也不需要修改付款状态，更不需要触发退款流程。即使这些资源都位于同一个 CRM 系统，访问策略也应把“读工单摘要”和“导出客户隐私数据”区分开。对代理来说，资源名相同并不意味着动作风险相同；目标系统相同也不意味着任务边界相同。

长期 API key 会掩盖代理路径变化

稳定 API key 的危险在于它把多次不同任务折叠成同一把钥匙。代理今天查 FAQ，明天分析客户隐私，后天触发业务流程，若全部沿用同一组长期权限，IAM 系统很难判断某次调用是否仍服务当前任务。Enhanced 阶段要求把钥匙拆到任务和动作层面。

5.4 实时检测：从事后审计走向运行时发现

Foundation 阶段已经把日志、跟踪和 SIEM 放进系统，目的是形成审计能力和合规证据。Enhanced 阶段在此基础上加入实时检测：系统要在流程运行时观察异常，判断行为链路或系统状态是否正在偏离预期。讲者提出的问题很直接：有没有事情正在出错？能否检测到异常？这个流程或系统的行为是否正在以错误方式运行？

这里的进步可以理解为摄像头从“录像留档”升级为“有人值守并能触发告警”。录像留档对事后调查有价值，可以帮助还原发生了什么；实时检测关注正在发生的偏离，能让后续 Adaptive 阶段触发更强认证、拒绝访问或实时撤销。对于代理式系统，这种时间差很关键。代理执行速度快，工具调用可能连续发生，若安全团队只在任务结束后阅读日志，很多越权动作已经完成。

异常检测看的是什么

实时检测可以观察多类信号：代理突然访问与任务无关的系统，调用频率显著升高，任务路径偏离历史模式，短时间内请求敏感数据，尝试执行高影响动作，或者凭证在异常上下文中被使用。这些信号未必都说明攻击正在发生，也可能来自错误配置、提示词诱导、工具选择失误或业务流程缺陷。检测机制的价值在于及时暴露偏离，让系统有机会介入。

实时检测也把审计证据变成运行时安全信号。日志仍然重要，因为没有日志就缺少事实基础；但 Enhanced 阶段要求日志进入可计算、可告警、可联动的检测链。若某个代理身份在任务凭证即将过期后仍尝试访问敏感资源，或者在客户服务任务中开始调用财务修改接口，系统应在运行时识别这种不协调，并把信号交给策略引擎、认证系统或撤销机制。

只留审计证据会错过拦截窗口

事后审计回答“后来怎样解释这件事”，实时检测回答“现在是否出现偏离”。代理式流程速度快、路径动态，单靠事后证据容易让异常在被发现前完成资源访问。Enhanced 阶段把检测提前到运行时，为下一阶段的实时撤销和风险感知认证提供触发信号。

5.5 本章小结

Enhanced 阶段把 Agentic IAM 从基础可追踪推进到主动治理。它首先让代理成为一等身份主体，使生命周期、策略、审计、治理、所有权和撤销都能直接落到代理本身；随后使用临时凭证，把访问能力限制在当前任务和极短宽限窗口内；接着通过细粒度上下文访问，把代理身份、任务、资源、上下文、风险、时间和目标动作纳入同一个访问决策；最后加入实时检测，把 Foundation 阶段留下的日志证据升级为运行时异常发现能力。

这一章的核心收获可以压缩成一句话：Enhanced 阶段缩小代理“可能做到的事”和“当前任

务允许做的事”之间的距离。传统 API key 与稳定应用权限适合较固定的调用路径，代理式系统的路径会动态变化，所以 IAM 必须把凭证、权限和检测都贴近任务现场。到了这个层次，代理从应用背后的模糊调用者升级为可治理、可限权、可审计、可撤销的运行时代身份主体。

6 第四步：Adaptive 的持续认证、风险感知与实时撤销

6.1 为什么最高阶段必须是自适应

讲者把第四步称为 Adaptive，核心原因来自代理式系统的运行形态。传统应用的调用路径通常比较稳定：应用从固定入口进来，调用少数已知接口，访问范围长期可预测。代理式流程的路径会随目标、工具选择、上下文和模型输出而变化；同一个业务目标在两次运行中可能走出不同步骤，甚至触发不同资源调用。这里的非确定性指的是：系统并未承诺每次都沿着完全相同的执行轨迹前进。用公式写，可以把一次代理流程看成一条随时间展开的路径：

$$\pi_i = (s_1, a_1, r_1), (s_2, a_2, r_2), \dots, (s_k, a_k, r_k)$$

其中， π_i 表示第 i 次运行形成的流程路径； s_j 表示第 j 个步骤的上下文状态； a_j 表示该步骤采取的动作，例如调用工具、查询数据库或触发业务接口； r_j 表示该动作触达的资源； k 表示本次流程实际经历的步骤数量。代理式系统的难点在于， π_1 和 π_2 即使服务同一个用户请求，也可能出现不同的 a_j 、 r_j 和 k 。因此，IAM 控制如果只在流程开头做一次授权，后续路径一旦偏离初始假设，系统就会进入盲飞状态。

Adaptive 的控制对象

Adaptive 成熟度关注运行中的每一步。它要求系统在流程进行时持续确认代理是谁、正在代表谁、要做什么、接触什么资源、风险是否上升，以及一旦检测到异常能否立刻撤销访问。它把身份验证、风险判断和阻断能力放进代理流程本身，同时避免把控制停留在启动前审批。

这个阶段承接前一章的实时检测。Enhanced 阶段已经让系统开始观察“是否有异常、行为是否偏离预期”。Adaptive 阶段继续追问：检测到了以后，访问是否还能继续？凭证是否还能使用？数据动作是否还能落地？如果答案仍然依赖人工排查或离线工单，代理的速度会压过治理速度，异常流程可能在几秒内完成不该完成的动作。

6.2 持续识别与持续认证：每一道敏感门都要验工牌

讲者在这一段反复强调“throughout the flow”和“each agent”。意思很直接：在流程发生的整个过程中，每一个代理都要持续识别自身，并持续完成认证，让系统知道当前动作究竟来自谁。这里的“识别”强调身份声明，例如这是哪个代理、属于哪个系统、代表哪个用户或任务；“认证”强调对身份声明进行验证，例如凭证是否有效、签名是否可信、会话是否仍处在允许范围内。

可以把这一机制想象成一栋办公楼。进入大楼大厅时刷一次工牌，只能说明来访者曾经通过入口检查。若来访者随后要进入财务室、研发机房、客户资料档案室，每一道敏感门都应再次检

查工牌、门禁规则和访问理由。代理流程也是这样：开头拿到身份并不意味着后续每一步都自动可信。流程越动态，敏感门越多，系统越需要在关键节点刷新信任。

持续认证与重复登录的区别

持续认证并不等同于让代理在每一步重新走一遍笨重登录流程。更合理的工程实现通常是短生命周期凭证、会话状态校验、签名验证、设备或工作负载身份验证、上下文检查、策略引擎决策等机制的组合。它的目标是让信任随步骤更新，避免“一次通过、全程通行”的通行证变成长期风险。

持续认证还会改变审计质量。若系统只记录“某个项目 API key 被使用”，事后只能看到粗粒度入口。若系统在每一步都保留代理身份、代表关系、凭证建立时间、访问资源、认证结果和上下文，审计人员就能还原代理行为链。对问责而言，这相当于把摄像头从楼门口延伸到关键房间门口，事后调查可以定位到具体门、具体人、具体时间和具体动作。

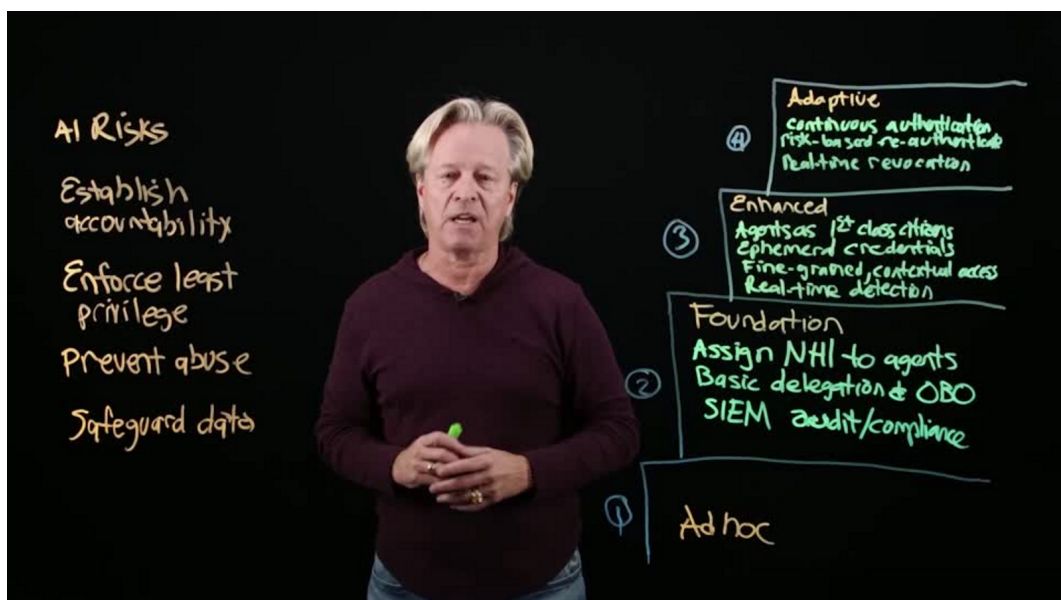


图 6: Adaptive 层包含持续认证、风险感知的重新认证和实时撤销；这些能力把检测结果变成运行时访问控制。⁵

6.3 风险感知认证：敏感数据会抬高门槛

Adaptive 的第二个关键机制是风险感知认证。讲者给出的例子是敏感数据访问：当流程开始接触更高敏感度的数据时，系统需要知道每一步代理流程包含多少风险，然后根据风险决定认证强度。换句话说，认证强度不应固定成一个静态等级；当动作更危险、数据更敏感、上下文更反常时，系统应该提高门槛，甚至拒绝继续。

可以把运行时风险写成一个函数：

$$\mathcal{R}_t = f(\text{sensitivity, action, context, anomaly, identity})$$

⁵视频画面时间区间：00:14:05–00:14:15。

其中， $\mathcal{R}_t$ 表示时间 t 上当前步骤的风险分数或风险等级； f 表示风险评估函数，也就是组织把多个因素组合成风险判断的方法；sensitivity 表示资源或数据的敏感度，例如公开文档、内部资料、客户隐私或财务数据；action 表示代理准备执行的动作，例如读取、导出、修改、删除、转发或触发动款；context 表示上下文，例如用户、任务目标、时间、网络位置、调用链和业务场景；anomaly 表示异常信号，例如行为突然偏离历史模式、调用频率异常、访问路径异常或检测系统已发出告警；identity 表示代理、用户、服务账号和委派关系等身份信息。

认证强度可以随风险上升而提高：

$$\mathcal{A}_t = g(\mathcal{R}_t)$$

其中， $\mathcal{A}_t$ 表示时间 t 上系统要求的认证强度或验证要求； g 表示把风险等级映射到认证动作的策略函数； $\mathcal{R}_t$ 仍然表示当前步骤风险。若 $\mathcal{R}_t$ 较低，系统可能只需要校验短期凭证和基本上下文；若 $\mathcal{R}_t$ 升高，系统可能要求额外签名、二次授权、人工批准、更严格的策略检查，或直接拒绝该动作。这个公式的价值在于提醒读者：Adaptive 要求认证负担贴近风险，避免到处加重认证负担。

敏感度只看数据分类会漏掉动作风险

同一份数据在不同动作下风险不同。读取一条客户记录、批量导出全部客户记录、把记录发送到外部系统、修改客户状态，这些动作的风险不在同一层级。风险感知认证需要同时看数据敏感度和动作含义。只看数据标签会低估批量导出、外发、写入和高影响业务触发带来的实际风险。

因此，Adaptive 的访问判断更像机场安检里的分层检查。普通登机口只需要常规证件核验；进入跑道控制区、危险品仓库或驾驶舱附近，系统会提高核验强度并检查进入理由。代理访问普通内部知识库和访问客户隐私库，不能共用同一把“通用钥匙”。流程每往高敏感区域走一步，IAM 都应重新计算风险并调整认证要求。

6.4 实时撤销：检测之后必须能阻断

Adaptive 的第三个关键机制是实时撤销。讲者的逻辑链很清楚：前一步引入实时检测，让系统能发现异常；第四步要在此基础上撤销访问、停止动作，并放置阻断。实时撤销要同时覆盖账号状态、运行中凭证、后续工具调用、敏感资源访问和高风险动作提交，让正在运行的凭证失效，让后续工具调用无法继续，让敏感资源拒绝当前代理链路，让高风险动作无法提交。

可以把检测到撤销的闭环表达成：

$$D_t = 1 \Rightarrow \text{revoke}(c_t), \text{block}(a_t, r_t)$$

其中， $D_t = 1$ 表示系统在时间 t 检测到异常或高风险事件； c_t 表示当时仍可使用的代理凭证； $\text{revoke}(c_t)$ 表示撤销该凭证或使它立即失效； a_t 表示代理准备执行或正在执行的动作； r_t 表示该动作触达的资源； $\text{block}(a_t, r_t)$ 表示阻断这次资源动作。这个表达式刻意把“发现”与“阻断”连在一起，因为只发现异常却无法阻断，在高速代理流程里常常来不及。

从检测到撤销的运行时闭环

实时检测回答“系统是否看到了异常”；实时撤销回答“系统是否能让异常停止”。成熟的代理式 IAM 需要把检测信号接入凭证、策略和资源控制面，让风险事件可以触发撤销、拒绝、隔离或降级，避免只在日志里留下事后证据。

现实落地时，撤销还要考虑传播延迟。若凭证撤销记录已经写入身份系统，但资源网关缓存仍允许旧凭证继续访问，阻断效果会被削弱。若代理已经把敏感数据取出并传给外部工具，再撤销凭证也无法收回已经外泄的数据。因此，Adaptive 的工程边界还包括短生命周期凭证、资源侧即时校验、缓存失效策略、出站数据控制和高风险动作提交前检查。未知盲点在这里很尖锐：很多团队以为“有撤销按钮”就等于有实时撤销能力，实际系统可能需要数十秒甚至数分钟才让撤销生效。

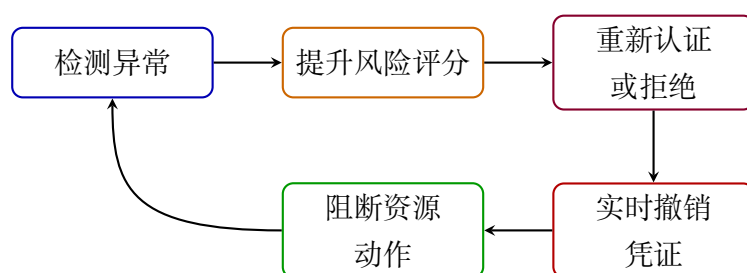


图 7: Adaptive 层的运行时闭环：检测、风险提升、重新认证、撤销与阻断必须能在流程仍在运行时改变访问结果。

6.5 本章小结

Adaptive 成熟度把代理式 IAM 推到运行时治理层面。由于代理流程动态且非确定，同一个目标可能在每次运行中触达不同动作和资源，系统需要在流程全程持续识别代理、持续认证身份、根据敏感度和上下文计算风险，并在检测到异常后实时撤销访问、阻断动作。它的关键价值在于闭环：身份持续可见，风险持续可算，访问持续可控，异常可以被立即压住。

7 四步控制如何回到四类风险

7.1 成熟度是累积阶梯

讲者在结尾前把四个成熟度阶段重新串起来：一开始几乎没有控制，随后加入 Foundation 的基础能力，再加入 Enhanced 的凭证、细粒度访问和检测能力，最后进入 Adaptive 的持续评估。这个顺序具有工程含义，描述的是能力依赖关系。没有代理身份，就难以做问责；没有委派链和日志，就难以解释代理代表谁；没有短期凭证和上下文访问，就难以限制每个步骤；没有实时检测，就难以触发实时撤销。

可以用集合增量表达这种累积关系：

$$C_n = C_{n-1} \cup \Delta C_n$$

其中, C_n 表示第 n 个成熟度阶段结束后系统已经具备的控制集合; C_{n-1} 表示上一阶段已经具备的控制集合; ΔC_n 表示本阶段新增的控制能力; $\cup$ 表示集合并集, 也就是保留原有控制并加入新增控制。这个公式强调成熟度提升靠叠加可运行控制完成。若前一阶段空心, 后一阶段会变成口号。

四级阶梯的直观读法

Ad Hoc 像临时搭起的工作台, 能让团队快速试验, 但风险边界模糊。Foundation 像给工作台接上门禁、登记簿和摄像头, 先知道谁在做事。Enhanced 像给每项任务发临时钥匙并限制房间范围, 开始按任务约束权限。Adaptive 像在每一道敏感门口加上动态安检和远程封锁, 一旦风险升高就提高检查或关闭通道。

7.2 问责: 把行为贴回具体代理与凭证

第一类风险是 accountability, 也就是问责或可追责。前文提到, 传统系统常把动作归到项目团队、API key、功能账号或系统账号上; 代理式系统中, 这种粗粒度归因不足以解释非确定路径。成熟度模型通过多个控制把问责补齐: Foundation 引入非人类身份, 让代理动作有身份锚点; 委派和 on-behalf-of 关系让系统知道代理代表谁行动; SIEM、日志和审计能力留下证据; Enhanced 让代理成为身份治理系统中的一等主体, 并使用短生命周期凭证; Adaptive 在流程全程持续认证, 让身份在每一步保持可验证。

这样形成的问责链可以读成一句工程问题: 在某个时间点, 哪个代理、拿着哪张短期凭证、代表哪个用户或任务、在什么上下文中、对哪个资源做了什么动作? 如果系统能回答这句话, 调查、合规和责任划分才有落点。短期凭证尤其重要, 因为凭证建立时间和失效时间会把责任窗口压缩到具体任务范围内, 避免长期 API key 把多次运行混成一团。

7.3 最小权限: 让代理只能做当前任务允许的事

第二类风险是 least privilege。讲者在结尾把它明确映射到 fine-grained contextual access: 代理只能做被允许做的事情。这句话看似简单, 真正难点在“当前任务”和“当前上下文”。如果代理正在查询内部 FAQ, 它不需要批量导出客户隐私; 如果代理正在整理工单, 它不需要修改付款状态; 如果代理代表一个普通用户行动, 它不能借系统身份绕过用户的数据边界。

成熟度阶梯对最小权限的贡献是逐层压缩权限窗口。Foundation 先让系统知道代理和委派关系, 避免权限无主; Enhanced 使用短期凭证和细粒度上下文访问, 让凭证只服务当前任务; Adaptive 在每一步重新评估上下文和风险, 当流程转向高敏感动作时提高认证强度或拒绝。结果是权限从“给这个应用一把长期钥匙”, 转向“在这个任务、这个时间、这个上下文、这个资源动作上给出可解释许可”。

最小权限的失败通常来自方便

为了让原型快速跑通, 团队容易给代理一个宽权限系统账号。短期看它减少集成成本, 长期看它制造了超级代理: 一个入口可以横向访问多类资源, 攻击者、错误提示词或开发失误都可能把影响面放大。最小权限要求把方便让位给边界, 把长期通行让位给任务级许可。

7.4 防滥用：把观察、检测和撤销接成响应链

第三类风险是 prevent abuse，即防止有意或无意的滥用。讲者把这一类风险映射到 SIEM、错误或异常检测，以及凭证的实时撤销。SIEM 提供集中可见性，日志和审计让系统有历史证据；实时检测关注正在发生的异常行为；实时撤销让系统在发现异常后能够收回访问并放置阻断。三者合起来，才从“看见问题”推进到“处理问题”。

这里要避免一个常见误读：防滥用并不只服务恶意攻击场景。代理式系统里的无意滥用同样高频，例如开发者把代理接到错误系统，策略配置过宽，或提示词让代理选择了不合适的工具。检测机制需要覆盖恶意、错误、偏离预期和异常上下文；撤销机制需要能在这些信号出现后及时介入。若系统只有告警邮件，代理仍能继续访问资源，安全团队看到的是事后报告，运行时控制仍然缺席。

7.5 保护数据：把身份传播、委派与风险判断合在访问边界上

第四类风险是 safeguard data，也就是保护数据。讲者把数据保护映射到身份传播、委派、on-behalf-of 关系以及代理识别。这些机制共同解决一个问题：数据访问决策不能只看“有没有一个有效系统凭证”，还要看用户是否有权、代理是否有权、代理是否真的代表该用户或任务、当前动作是否符合上下文。

数据保护的访问边界像双层闸门。第一层确认用户或业务主体有资格接触数据；第二层确认代理在当前任务中可以执行该类数据动作。Adaptive 又给闸门加上风险感知：当访问对象从普通资料变成客户隐私、财务记录或安全配置，系统应提高认证强度并保留更细证据。这样，数据保护不再只依赖静态 ACL 或角色表，还把代理身份、委派链、动作语义、上下文和异常信号放进同一个决策面。

7.6 本章小结

四步成熟度模型的价值在于把控制重新落回四类风险。问责依赖代理身份、短期凭证、委派链和审计证据；最小权限依赖任务级凭证、细粒度上下文访问和逐步评估；防滥用依赖 SIEM、实时检测和实时撤销；保护数据依赖用户授权、代理授权、身份传播、on-behalf-of 语义和风险感知认证。成熟度阶梯每升一级，都应让某一类风险变得更可见、更可限、更可查、更可阻断。

8 总结与实施路线

8.1 从基础开始，沿成熟度模型提高

讲者最后给出的建议很务实：团队应先从 basics 入手，建立 foundation，然后沿着成熟度模型继续提高系统。它没有要求组织一次性堆满所有能力。这个提醒很重要，因为代理式 IAM 的能力清单很容易吓人：身份、委派、SIEM、短期凭证、上下文访问、实时检测、持续认证、实时撤销，每一项都涉及系统集成和治理流程。若团队试图同时铺开，常见结果是概念很多、落地很薄、责任边界仍然模糊。

更稳妥的实施路线可以按“先看见，再限制，再响应，再自适应”推进。第一步是盘点代理和代理式工作流，弄清楚哪些代理能调用工具、连接哪些系统、代表哪些用户或业务任务。第二步是给代理分配非人类身份，让代理活动不再藏在项目账号、API key 或系统账号背后。第三步是建立委派和 on-behalf-of 语义，明确代理代表谁行动，以及自主代理在什么边界内行动。第四步是把代理活动接入 SIEM、日志、审计和合规流程，让系统先具备可见性。

随后进入权限收缩和运行时控制。第五步是引入短生命周期凭证，把凭证有效期贴近任务有效期。第六步是做细粒度上下文访问决策，把资源、动作、时间、委派链和任务目标纳入策略。第七步是部署实时检测，观察异常行为、错误路径和偏离预期的流程。第八步是把检测接到持续认证、风险感知认证和实时撤销，让风险信号能触发更强认证、拒绝访问、撤销凭证或阻断资源动作。

最终压缩

面向代理式系统的 IAM 是运行时治理问题，同时包含账号供给。它要管理代理是谁、代表谁、拿什么凭证、在什么上下文中触达什么资源，并且要在流程变化时持续更新判断。未来防护能力来自分阶段成熟：先建立身份和可见性，再收缩权限窗口，再加入检测响应，最后让认证、风险和撤销随流程动态变化。

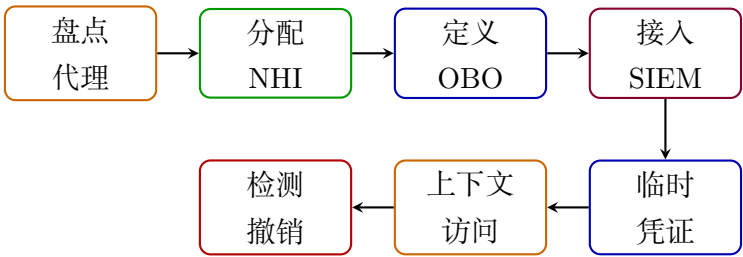


图 8: 实施路线：先建立可追踪身份与审计基础，再逐步引入任务级凭证、上下文访问和实时撤销。

8.2 实践中的优先级与风险

实施时最容易低估的风险有三类。第一类是身份库存不完整：组织可能只登记了面向用户的应用，却漏掉脚本、自动化代理、实验性助手和内部工具代理。第二类是委派语义含糊：系统知道代理在行动，却不知道它代表谁、被谁授权、能否脱离用户继续执行。第三类是撤销链路过慢：检测系统已经告警，凭证和资源侧仍允许旧访问继续发生。

这些风险会让成熟度模型出现“纸面成熟”。例如，团队声称已经有日志，但日志只记录系统账号；声称已经有最小权限，但代理仍持有长期宽权限 API key；声称已经有实时检测，但撤销动作需要人工工单；声称已经保护数据，但访问决策只检查用户授权，没有检查代理授权。真正的落地标准应看运行时证据：每个关键动作能否追到代理身份，凭证是否随任务收束，策略是否理解上下文，检测是否能触发阻断，数据访问是否同时满足用户和代理两侧授权。

8.3 本章小结

这套四步模型把未来防护拆成可执行路线: Ad Hoc 暴露低控制状态, Foundation 建立身份、委派和审计基础, Enhanced 引入一等代理身份、短期凭证、细粒度上下文访问和实时检测, Adaptive 在动态流程中持续认证、计算风险并实时撤销。最终目标是让代理式系统在速度提高的同时保持可追责、可限权、可响应、可守住数据边界。团队应从基础控制开始, 把每一级新增能力落到可运行机制, 再沿成熟度阶梯逐步提高, 避免用一张功能清单替代治理闭环。